

TPIDS — Proyecto Final

Tienda full-stack con autenticación

Catálogo de **productos, categorías y usuarios**
con registro, login JWT, verificación de email y reset de contraseña.

Angular 21 · NestJS 11 · PostgreSQL · Servidor MCP

Agenda

1. Arquitectura general
2. Backend — NestJS + TypeORM + Postgres
3. Frontend — Angular 21
4. Servidor MCP
5. Cómo levantar el proyecto
6. Variables de entorno

1. Arquitectura general

```
practico final/  
├── back/      API REST – NestJS 11 + TypeORM + PostgreSQL  
├── front/     SPA – Angular 21 (standalone components)  
├── mcp/       Servidor MCP (herramientas sobre la API)  
└── docs/      Documentación
```

Capa	Tecnología	Puerto
Frontend	Angular 21 (<code>ng serve</code>)	4200
Backend	NestJS 11 (Express)	3000
Base de datos	PostgreSQL 16 (Docker)	5433 → 5432

El front habla con el back por HTTP; el back persiste en Postgres; auth por **JWT Bearer** .

2. Backend — Stack

- **NestJS 11** (platform-express) + **TypeORM** + driver **pg**
- **Auth:** `@nestjs/jwt` + `@nestjs/passport` + `passport-jwt`
- **Hash:** `bcrypt` (rounds configurable `BCRYPT_COST`, default 12)
- **Mail:** `nodemailer` (SMTP, Mailtrap sandbox)
- **Validación:** `class-validator` + `class-transformer`
con `ValidationPipe` `global(whitelist, forbidNonWhitelisted, transform)`

2. Backend — Módulos (`back/src/`)

- **auth/** — registro, login, verificación email, reset password. JWT + Passport, `@Roles` .
- **users/** — cuenta propia (borrar, cambiar password/email) y roles (admin).
- **products/** — CRUD + filtros, orden y paginación (patrón Repository).
- **categories/** — CRUD + productos por categoría.
- **placeholder-users/** — datos externos (JSONPlaceholder) o locales (`USERS_SOURCE`).
- **mail/** — envío interno de emails (verificación / reset).
- **common/** — middlewares `Logger` y `Timing` (`X-Response-Time`), paginación.

2. Backend — Modelo de datos

- **users:** `id (uuid)`, `email (único, normalizado)`, `passwordHash (select:false)`,
`role (user|admin)`, `isVerified`, `tokens de verificación/reset (select:false)`
→ **el primer usuario registrado queda ADMIN**
- **categories:** `id`, `name`
- **products:** `id`, `name`, `price (real)`, `stock (int, default 0)`,
`categoryId` → **ManyToOne** a `categories (eager: true)`

2. Endpoints — /auth

Método	Ruta	Auth
POST	/auth/register	público
POST	/auth/login	público
GET	/auth/me	JWT
POST	/auth/verify-email	público
POST	/auth/resend-verification	JWT
POST	/auth/forgot-password	público
POST	/auth/reset-password	público

2. Endpoints — /users

Todos requieren **JWT**:

Método	Ruta	Rol
DELETE	/users/me	user
PATCH	/users/me/password	user
PATCH	/users/me/email	user
GET	/users	admin
PATCH/PUT	/users/:id/role	admin

2. Endpoints — `/products` y `/categories`

Públicos (sin guard) — CRUD completo:

- `GET /products?name=&sortBy=&order=&page=&limit=` → listado paginado
- `PATCH /products/:id/stock` → descuenta stock validando disponibilidad
- `GET /categories/:id/products` → productos de la categoría (paginado)
- `DELETE /categories/:id` → falla si tiene productos asociados

⚠ Escritura abierta en la API: la restricción "solo admin edita" hoy es solo de UI.

2. Autenticación (flujo)

- **Registro** → hash bcrypt, genera `verificationToken`, envía email. 1er user = ADMIN.
- **Login** → `bcrypt.compare`, firma JWT `{ sub, role }`, expira en `JWT_EXPIRES_SEC` (1h).
- **JwtStrategy** → extrae el Bearer, valida el secret, recarga el usuario desde DB.
- **Reset password** → token con expiración 1h por email (`FRONTEND_URL/reset-password?token=`).
- **Cambio de email** logueado → `isVerified=false` y re-verificación.

2. Mail y configuración notable

Mail — `MailService` (Nodemailer/SMTP, env `MAIL_*`)

`sendVerificationEmail` / `sendPasswordResetEmail`. Fallos de SMTP se loguean **sin romper** el request. `forgot-password` nunca revela si el email existe.

Config

- `synchronize: true` → TypeORM crea/actualiza el esquema en cada arranque.
- CORS restringido a `CORS_ORIGIN`, `credentials: true`.
- Campos sensibles con `select: false`.
- Paginación: `{ data, meta: { page, limit, total, totalPages } }` (default 10 / máx 50).

3. Frontend — Stack

- **Angular 21, standalone components** con lazy loading (`loadComponent`)
- **Bootstrap 5.3, RxJS**, formularios reactivos
- Estado por **signals** (sin NgRx/Akita) — local o en servicios

Estructura (`front/src/app/`)

`pages/` · `services/` (auth, products, categories, users) · `guards/` (auth, admin)
`interceptors/` (jwt) · `shared/` (navbar, bottom-nav, footer, toast, bottom-sheet)

3. Frontend — Rutas

Ruta	Guard
/ · /login · /register	—
/verify-email · /forgot-password · /reset-password	—
/verify-pending	authGuard
/profile	authGuard
/products · /products/:id · /categories	authGuard
/admin/users	authGuard + adminGuard

3. Frontend — Auth y UX

Auth

- **AuthService** guarda el JWT en `localStorage (access_token)`, `user = signal<SafeUser>`.
- **jwtInterceptor** agrega `Authorization: Bearer` a toda request hacia `apiUrl`.
- **authGuard** → `/login`; **adminGuard** → `/`.
- CRUD visible para todos, pero **solo admin** crea/edita/borra (bottom-sheet).

UX

- **ToastService** (signals): `success | error | info`, auto-dismiss 4s.
- Errores por página: `firstValueFrom()` + `try/catch` → toast (sin interceptor global).

4. Servidor MCP (mcp/)

Bridge **Model Context Protocol** que expone la API REST como **tools** para clientes LLM.

- **SDK** `@modelcontextprotocol/sdk` (`McpServer` + transporte **stdio**)
- **axios** como cliente HTTP, **tsx** para correr TypeScript

Estructura (mcp/src/)

- `index.ts` — crea el server `api-c-bridge`, registra tools, **auto-login**, conecta por stdio
- `api-client.ts` — singleton sobre axios, guarda el JWT e inyecta `Bearer`
- `tool-factory.ts` — registra tools con `inputSchema` (**zod**), serializa y mapea errores
- `tools/` — `auth`, `products`, `categories`, `users`

4. MCP — Herramientas (18)

Grupo	Tools
auth	<code>auth_login</code> , <code>auth_register</code> , <code>auth_me</code> , <code>delete_my_account</code>
products	<code>list_products</code> , <code>get_product</code> , <code>create_product</code> , <code>update_product</code> , <code>delete_product</code>
categories	<code>list_categories</code> , <code>get_category</code> , <code>create_category</code> , <code>update_category</code> , <code>delete_category</code>
users	<code>list_users</code> , <code>update_user_role</code> , <code>update_my_password</code> , <code>update_my_email</code>

Auth: auto-login con `API_C_EMAIL` / `API_C_PASSWORD` , o `auth_login` en runtime.
Token en memoria, enviado como `Bearer` . Habla por **stdio** vía cliente MCP.

5. Cómo levantar el proyecto

```
# 1. Base de datos
cd "practico final/back"
docker compose up -d          # Postgres en localhost:5433

# 2. Backend
npm install
npm run seed                  # datos de prueba (opcional)
npm start                     # API en http://localhost:3000

# 3. Frontend
cd ../front
npm install && npm start      # SPA en http://localhost:4200
```

back/.env debe tener **DB_PORT=5433** (el compose mapea 5433:5432).

5. Datos de prueba — `npm run seed`

4 categorías · 9 productos · 2 usuarios verificados (idempotente)

Email	Password	Rol
<code>admin@tpids.local</code>	<code>Admin123!</code>	admin
<code>user@tpids.local</code>	<code>User123!</code>	user

6. Variables de entorno (`back/.env`)

Variable	Descripción
<code>BCRYPT_COST</code>	Rounds de bcrypt (default 12)
<code>DB_HOST/PORT/USER/PASSWORD/NAME</code>	Conexión Postgres (<code>DB_PORT=5433</code> con Docker)
<code>JWT_SECRET</code>	Secreto de firma JWT (obligatorio)
<code>JWT_EXPIRES_SEC</code>	Expiración del token (default 3600)
<code>CORS_ORIGIN</code> / <code>FRONTEND_URL</code>	Origen del front / base para emails
<code>MAIL_HOST/PORT/USER/PASS/FROM</code>	SMTP (Mailtrap sandbox)

¡Gracias!

TPIDS — Angular 21 · NestJS 11 · PostgreSQL · MCP

Documentación completa en `docs/INFORME-PROYECTO.md`